Conclusion to Full-Stack Development

(https://github.com/learn-co-curriculum/python-p4-conclusion-to-full-stack-development) (https://github.com/learn-co-curriculum/python-p4-conclusion-to-full-stack-development/issues/new)

Learning Goals

- Use React and Flask together to build beautiful and powerful web applications.
- Organize client and server code so that it is easy to understand and maintain.
- Create websocket connections between clients and servers to improve our applications' performance.

Key Vocab

- **Full-Stack Development**: development of a frontend and a backend for an application. True full-stack development includes a database, a logic/server layer, and a frontend built in JavaScript, HTML, and CSS.
- **Backend**: the layer of a full-stack application that handles business logic and other programmatic tasks that users do not or should not see. Can be written in many languages, including Python, Java, Ruby, PHP, and more.
- **Frontend**: the layer of a full-stack application that users see and interact with. It is always written in the frontend languages: JavaScript, HTML, and CSS. (*There are others now, but they are based on these three.*)
- **Cross-Origin Resource Sharing (CORS)**: a method for a server to indicate any ports (or other identifiers) for servers that can share its resources.
- **Transmission Control Protocol (TCP)**: a protocol that defines how computers send data to each other. A connection is formed and stays active until the applications on either end have finished sending data to one another.
- **Hypertext Transfer Protocol (HTTP)**: a stateless protocol where applications communicate for the length of time that it takes for data to be transferred.
- **WebSocket**: a protocol that allows clients and servers to communicate with one another in both directions. The bidirectional nature of websocket communication allows a connected state to be generated and the connection maintained until it is terminated by one side. This allows for speedy and seamless connections between frontends and backends.

Conclusion

Most of your experience with web applications is with their *frontends*. The posts on Reddit, the scrolling thumbnails on Netflix, the map and tiny moving cars on Lyft- all of these are designed with JavaScript. It can be frustrating sometimes when learning backend languages and frameworks to know that the JavaScript is what everyone is seeing and interacting with.

That being said, none of these applications would do very much without a backend- specifically, in the case of Reddit, Netflix, and Lyft, they would not do very much without Flask.

Full-stack development gives software engineers the ability to design applications that do as much as they possibly can. They can be as visually appealing as Airbnb and as powerful as AWS (if you have the DS&A know-how). Use of ORMs and clever implementation of concepts like REST and WebSocket will expand the possibilities for your career even further.

In the next module, you will learn how to deploy your full-stack applications to the internet.

Resources

- **What is full stack development? - GeeksforGeeks (https://www.geeksforgeeks.org/what-is-full-stack-development/)**
- **Flask - Pallets (https://flask.palletsprojects.com/en/2.2.x/)**
- **Getting Started - React (https://reactjs.org/docs/getting-started.html)**
- **Introduction - Socket.IO (https://socket.io/docs/v4/)**